A Mini Project Report on

DETECTING PLAGIARISM IN RESEARCH PAPERS

Submitted to

Jawaharlal Nehru Technological University, Hyderabad in partial fulfilment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

KASANI VINAY KUMAR	**22831A0591**
JUPELLI MANEESH	**22831A0578**
JANGALA SINDHU	**22831A0573**
KONDAMALLA SRUCHEN KUMAR	**22831A0595**

Under the Esteemed Guidance of

Dr. B. Santhosh Kumar

Professor & Head – Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

GURU NANAK INSTITUTE OF TECHNOLOGY

(Affiliated to JNTUH-Hyderabad)

Ranga Reddy District-501506

MAY-2025

GURU NANAK INSTITUTE OF TECHNOLOGY
(Affiliated to JNTUH-Hyderabad)
Ranga Reddy District-501506

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the Mini-project entitled DETECTING PLAGIARISM IN RESEARCH PAPERS is being submitted by Mr. Kasani Vinay Kumar bearing Roll No. 22831A0591, Mr. Jupelli Maneesh bearing Roll No. 22831A0578, Ms. Jangala Sindhu bearing Roll No. 22831A0573, Mr. Kondamalla Sruchen Kumar bearing Roll No. 22831A0595 in partial fulfillment for the award of the Degree of Bachelor of the Technology in COMPUTER SCIENCE AND ENGINEERING to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by them under my guidance and supervision.

The results embodied in this Mini-project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

DR. B. SANTHOSH KUMAR
INTERNAL GUIDE

Dr. B. SANTHOSH KUMAR
HEAD OF DEPARTMENT

EXTERNAL EXAMINER

DECLARATION BY STUDENTS

We hereby declare that the major project report entitled **"Detecting Plagiarism in Research Papers"** is the work done by **Kasani Vinay Kumar, Jupelli Maneesh, Jangala Sindhu, Kondamalla Sruchen Kumar** bearing the roll no's **22831A0591, 22831A0578, 22831A0573, 22831A0595** towards the fulfilment of the requirement for the award of the **Degree of Bachelor of Technology in Computer Science and Engineering**, to **Jawaharlal Nehru Technological University, Hyderabad,** is the result of the work carried out under the guidance **Dr. B Santhosh Kumar**, Professor & Head – Department of Computer Science and Engineering **Guru Nanak Institute of Technology, Hyderabad.**

I confirm the following:

1. I have contributed the entire project process, from the selection of the project title to the submission of the final report.
2. The project report has been prepared as per guidelines, ensuring adherence to high standards of quality, clarity, and structure.
3. All information provided and work done is completely genuine and is not copied from any online or offline platform.
4. That I or any of my known have not submitted this project to any other University or College.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

22831A0591	**22831A0578**	**22831A0573**	**22831A0595**
K Vinay Kumar	**J Maneesh**	**J Sindhu**	**K Sruchen Kumar**

DECLARATION BY GUIDE

I, Dr. B. Santhosh Kumar, hereby declare that I have guided the minor project titled “Detecting Pagiarism in Research Papers” undertaken by K. Vinay Kumar (22831A0591), J. Maneesh (22831A0578), J. Sindhu (22831A0573), K. Sruchen Kumar (22831A0595). This project was carried out towards the fulfillment of the requirements for the award of the Degree of B.Tech in Computer Science and Engineering Guru Nanak Institute of Technology.

As the guide, I confirm the following:

5. I have overseen the entire project process, from the selection of the project title to the submission of the final report.
6. I have reviewed and certified the accuracy and relevance of the results presented in the report.
7. The work carried out is original, free from plagiarism, and adheres to ethical guidelines.
8. The contributions of each student have been appropriately recognized and assessed.
9. The project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of Guide: Dr. B. Santhosh Kumar Signature of the Guide

Date: 30/05/2025

Place:

Name of HOD: Dr. B. Santhosh Kumar Signature of the HOD

Department: Computer Science and Engineering

Date: 30/05/2025

Place:

ACKNOWLEDGEMENT

"Task successful" makes everyone happy. But the happiness will be gold without glitter if we didn't state the persons who have supported us to make it a success.

We would like to express our sincere thanks to **Dr. B. SANTHOSH KUMAR**, Professor & Head **-** Department of Computer Science and Engineering, Guru Nanak Institute of Technology for having guided me in developing the requisite capabilities for taking up this project.

We thank Project Coordinator **Mr. G. LAKPATHI**, Assistant Professor **CSE, GNIT** for providing seamless support and right suggestions that are given in the development of the project.

We specially thank our internal guide **Dr. B. SANTHOSH KUMAR**, Professor & Head – Department of Computer Science and Engineering for his constant guidance in every stage of the project. We would also like to thank all our lecturers for helping me in every possible way whenever the need arose.

On a more personal note, we thank our beloved parents and friends for their moral support during the course of our project.

GURU NANAK INSTITUTE OF TECHNOLOGY
(Affiliated to JNTUH-Hyderabad)
Ranga Reddy District-501506

CO-PO MAPPING

COs	Course Outcome	BL
CO1	Independently carry out literature survey in identified domain, and consolidate it to formulate a problem statement	4
CO2	Apply identified knowledge to solve a complex engineering problem and design a solution, implement and test the proposed solution	3
CO3	Use synthesis/modeling to simulate and solve a problem or apply appropriate method of analysis to draw valid conclusions and present, demonstrate, execute final version of project	3
CO4	Contribute effectively as a team member or leader to manage the project timeline and Incorporate the social, environmental and ethical issues effectively into solution of an engineering problem	6
CO5	Write pertinent project reports and make effective Project Presentations	3

COs	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO 1	PSO 2	PSO 3
CO 1	–	3	–	–	–	–	–	–	–	–	–	3	3	3	3
CO 2	3	–	3	–	–	–	–	–	–	–	–	–	3	3	3
CO 3	–	–	–	3	3	-	-	-	–	–	–	–	3	3	3
CO 4	–	–	–	–	–	3	3	3	3	–	3	–	–	3	–
CO 5	–	–	–	–	–	–	–	–	–	3	–	–	–	3	–

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

VISION

To be recognized as a leading department of Computer Science and Engineering in the region by students, employers and be known for leadership, Ethics, and commitment to fostering quality teaching-learning, research, and innovation.

MISSION

- Nurture young individuals into knowledgeable, skill-full and ethical professionals in their pursuit of Computer Science and Engineering.
- Nurture the faculty to expose them to world-class infrastructure.
- Sustain high performance by excellence in teaching, research and innovations.
- Extensive partnerships and collaborations with foreign universities for technology upgradation.
- Develop Industry-Interaction for innovation and product development.

ABSTRACT

Plagiarism is a growing concern in academic, research, and professional domains where originality of content is fundamental to maintaining credibility and ethical standards. As digital content becomes more accessible and widespread, traditional plagiarism detection methods—such as manual review and simple keyword matching—are no longer sufficient to detect complex instances of copied or reworded content. This project presents an automated and intelligent Plagiarism Detection System that provides a robust solution for identifying textual similarities across multiple documents.

The system is developed using Python and leverages powerful open-source libraries such as spaCy for Natural Language Processing (NLP), scikit-learn for vectorization and similarity measurement, and python-docx for handling PDF and DOCX file formats. It supports .txt, .pdf, and .docx documents, ensuring wide usability in academic institutions and professional settings.

After extracting content from the documents, the system performs preprocessing steps like lowercasing, tokenization, stopword removal, and punctuation filtering to ensure uniformity and relevance. It then applies TF-IDF (Term Frequency–Inverse Document Frequency) vectorization to represent documents numerically based on word significance. Using the cosine similarity algorithm, the system calculates pairwise similarity scores between documents and flags those exceeding a predefined threshold (e.g., 60%) as potentially plagiarized.

This modular and automated approach significantly improves the efficiency, scalability, and accuracy of plagiarism detection. It reduces manual effort, delivers consistent results, and supports decision-making through structured similarity reports. The system can be integrated into academic platforms, publishing workflows, or corporate review systems. It also sets a foundation for future enhancements, such as semantic similarity analysis, cross-language detection, and AI-based paraphrasing recognition.

TABLE OF CONTENTS

.

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

S.No	Term / Abbreviation	Full Form / Description
1	AI	Artificial Intelligence
2	CSV	Comma-Seperated Value
3	DOCX	Document Open XML
4	DFD	Data Flow Diagram
5	NLP	Natural Language Processing
6	PDF	Portable Document Format
7	POS	Part of Speech
8	TF	Term Frequency
9	TF-IDF	Term Frequency – Inverse Document Frequency
10	UI	User Interface
11	UML	Unified Modeling Language
12	XML	Extensible Markup Language

CHAPTER-1

INTRODUCTION

1.1 GENERAL

Plagiarism is a major issue in academic, research, and professional settings, where originality and authenticity play a crucial role in maintaining integrity. With the rapid growth of digital content, plagiarism has become easier to commit but harder to detect using traditional methods. Manual plagiarism detection, such as reading and comparing documents side by side, is time-consuming, labor-intensive, and often ineffective, especially when dealing with large volumes of text or sophisticated paraphrasing techniques. This necessitates the development of an automated plagiarism detection system that can efficiently and accurately analyze documents for similarities.

An automated plagiarism detection system offers a systematic and technology-driven approach to identifying copied or highly similar content between different documents. The system processes various document formats, extracts textual content, and applies analytical techniques to measure similarity. Unlike simple keyword searches, modern plagiarism detection systems utilize advanced text processing techniques that enable them to detect verbatim copying, reworded text, and even partial similarities that might otherwise go unnoticed.

The need for such a system is evident across multiple domains:

Academia: Educational institutions rely on plagiarism detection to maintain academic integrity, ensuring that students and researchers produce original work.

Publishing Industry: Journals and media outlets must verify the originality of submitted content to prevent copyright violations and maintain credibility.

Corporate and Legal Sectors: Businesses, law firms, and content creators require plagiarism detection to protect intellectual property and prevent unauthorized content duplication.

A well-designed plagiarism detection system provides several key benefits:

Accuracy and Reliability – The system detects not only direct copying but also paraphrased or modified content, improving the precision of plagiarism detection.

Time and Labor Efficiency – Automated analysis allows for rapid processing of multiple documents, significantly reducing the time and effort required for manual checking.

Scalability – The system can handle large datasets, making it suitable for universities, publishing houses, and organizations dealing with extensive textual data.

Actionable Insights – The system generates structured reports with similarity scores, enabling educators, researchers, and administrators to make informed decisions.

Ethical Writing and Intellectual Property Protection – By discouraging plagiarism and promoting originality, the system fosters ethical writing practices in academic and professional environments.

This project aims to develop an automated, scalable, and efficient plagiarism detection system that ensures originality in written work by systematically analyzing and comparing documents. By leveraging modern advancements in text analysis, the system will assist in preventing intellectual property violations, maintaining academic honesty, and streamlining content verification processes. Future enhancements may include support for multiple languages, integration with online databases, and deeper contextual analysis to further improve detection accuracy.

In conclusion, an automated plagiarism detection system plays a vital role in safeguarding originality, upholding ethical writing standards, and enhancing efficiency in content evaluation. Its integration into academic and professional workflows ensures that individuals and organizations can reliably assess document originality and take appropriate action against potential violations.

1.2 FEASIBILITY STUDY

The feasibility study helps determine whether the proposed plagiarism detection system is practical, cost-effective, and efficient to implement. It is evaluated under several categories to assess the strengths and limitations of the system design and technology used.

1.2.1 Types of Feasibility Study

The types of feasibility relevant to this project are:

- **Technical Feasibility:** The system is implemented using Python and libraries like spaCy, scikit-learn, and PyMuPDF. These tools provide robust support for text processing and similarity detection, making technical execution achievable.

- **Economic Feasibility:** The use of free, open-source tools eliminates licensing costs. The system runs on standard hardware with minimal resource usage, ensuring a low-cost solution.
- **Operational Feasibility:** The project meets a real-world need—detecting plagiarism in academic documents—and is easy to use for instructors and evaluators.
- **Legal Feasibility:** The system processes only user-provided documents and respects intellectual property rights.
- **Time Feasibility:** The entire project was completed within the academic project timeline, with a modular design allowing for efficient testing and future scalability.

1.2.2 NLP-Based Text Similarity Detection

Unlike deep learning-based models, this project adopts a classical Natural Language Processing (NLP) approach for textual similarity analysis. Key techniques used include:

- Text normalization using spaCy
- TF-IDF vectorization to numerically represent document content
- Cosine similarity to compare documents and compute overlap

This approach is effective for detecting lexical and partially paraphrased similarities in medium-sized datasets and can be run efficiently without requiring GPUs or advanced computing resources.

1.2.3 NLP vs Deep Learning in Plagiarism Detection

Feature	**NLP-Based (Used in this Project)**	**Deep Learning-Based (Alternative)**
Implementation Complexity	Simple and fast	Requires complex architecture and tuning
Resource Requirements	Low (CPU)	High (GPU, memory)
Performance on Small Datasets	Very effective	May overfit or underperform
Semantic Understanding	Limited	Better (can detect paraphrasing)
Training Data Needs	Not required	Requires large labeled datasets
Suitability for Academic Tools	High	Depends on infrastructure

CHAPTER-2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

1. Title: Existing Plagiarism Detection Techniques: A Systematic Mapping of the Scholarly Literature

Authors: Eisa, T.A.E., Salim, N., Alzahrani, S.

Year: 2015

This foundational paper offers a comprehensive systematic review of more than 60 plagiarism detection systems developed over the years. It classifies the detection methods into extrinsic (comparing against a reference corpus) and intrinsic (analyzing writing style shifts within a document) categories. Key similarity measures explored include fingerprinting, string matching, n-grams, and semantic analysis. The authors critically examine the limitations of current approaches, particularly in detecting obfuscated, disguised, or translated plagiarism. The paper emphasizes the need for future tools to integrate cross-lingual detection capabilities, semantic reasoning, and adaptive learning techniques. It also presents a timeline analysis of technological evolution in this domain and advocates for benchmarking frameworks and multilingual corpora to better evaluate detection performance.

2. Title: Plagiarism Detection Using Machine Learning

Authors: Kamat, O., Ghosh, T., Kalaivani, J., Angayarkanni, V., Rama, P.

Year: 2024

This recent study proposes a machine learning-based system leveraging Natural Language Processing (NLP) techniques for effective plagiarism detection. The framework integrates text preprocessing, feature extraction (TF-IDF, word embeddings), and classification using Logistic Regression and Support Vector Machines (SVM). Unlike traditional systems that rely on direct phrase matching, this model can detect paraphrased and restructured content by capturing semantic and syntactic patterns. Experiments conducted on real-world academic datasets demonstrate high F1-scores, indicating strong precision and recall. Additionally, the paper discusses the impact of feature engineering, overfitting risks, and generalization across different domains. The authors suggest future incorporation of deep learning models like LSTMs or Transformers for improved context retention.

3. Title: BERT-Enhanced Retrieval Tool for Homework Plagiarism Detection System
Authors: Xian, J., Yuan, J., Zheng, P., Chen, D., Yuntao, N.
Year: 2024

This paper introduces a cutting-edge plagiarism detection framework powered by BERT embeddings for semantic textual similarity. By leveraging contextual embeddings, the system excels at identifying paraphrased and semantically similar content, even when the structure and vocabulary are changed. Faiss (Facebook AI Similarity Search) is used for scalable and efficient nearest neighbor search, enabling real-time detection in large academic repositories. The tool achieves over 98% accuracy on benchmark datasets and includes a demo application to illustrate its usability for educators and institutions. Furthermore, the paper explores optimization techniques like embedding caching and vector indexing, ensuring both speed and accuracy. Its m: Existing Plagiarism Detection Techniques: A Systematic Mapping of the Scholarly Literature odular architecture enables easy integration into Learning Management Systems (LMS).

4. Title: An Effective Text Plagiarism Detection System Based on Feature Selection and SVM Techniques
Authors: Not explicitly specified (Springer Journal Publication)
Year: 2023

This study presents a hybrid plagiarism detection model combining semantic similarity measures from WordNet (Leacock-Chodorow, Wu-Palmer, Resnik) with SVM-based classification. Feature selection techniques are used to reduce dimensionality and improve model interpretability. The authors demonstrate that selecting high-weighted semantic features significantly enhances the detection of paraphrased and contextually modified content. The model is validated using academic text datasets and shows robust performance across subjects. The paper also compares the model with baseline methods, showing improved precision and reduced false positives. Future directions include integrating contextual embeddings and expanding support for multi-document comparisons.

5. Title: A First Step Towards Content Protecting Plagiarism Detection

Authors: Ihle, C., Schubotz, M., Meuschke, N., Gipp, B.

Year: 2020

This innovative research proposes a privacy-preserving plagiarism detection system based on Private Set Intersection (PSI), a cryptographic protocol allowing comparisons without revealing the actual content. The framework addresses a critical issue: academic institutions' reluctance to share sensitive documents for cross-checking due to data privacy concerns. The system ensures GDPR compliance and is particularly beneficial for collaborative plagiarism detection across institutions. Performance evaluations show that the solution scales well and maintains acceptable computational overhead. The paper also discusses trade-offs between cryptographic strength and system efficiency, and proposes enhancements for future versions using homomorphic encryption and federated learning.

6. Title: Improved Document Plagiarism Detection Using Semantic Analysis

Authors: Foltynek, T., Glendinning, I., Rosso, P.

Year: 2019

This work investigates the performance of popular commercial tools like Turnitin and Urkund from a semantic detection perspective. It underscores these systems' limitations in capturing deep or structural plagiarism and suggests that incorporating ontologies, thesauri, and AI-driven semantic parsers can address these gaps. The paper proposes a modular architecture for semantic plagiarism detection, emphasizing cross-lingual analysis, paraphrase recognition, and concept mapping. Ethical concerns, including false positives, student profiling, and the impact of algorithmic bias, are also discussed in depth. The study serves as a call to action for more transparent and pedagogically aligned plagiarism detection technologies.

7. Title: A Comparative Study of Text Similarity Detection Algorithms for Plagiarism Detection

Authors: Gupta, P., Garg, R., and Singh, S.

Year: 2022

This comparative analysis evaluates a wide array of similarity detection algorithms including Cosine Similarity, Jaccard Index, Euclidean Distance, TF-IDF, and Latent Semantic Indexing (LSI). The paper conducts both qualitative and quantitative assessments to understand the strengths and weaknesses of each approach in detecting literal and paraphrased plagiarism. Hybrid models that blend lexical, syntactic, and semantic components are shown to outperform isolated techniques in terms of recall and false-positive reduction. The authors also introduce a scalable experimental setup for testing models on academic, legal, and open-source datasets. Recommendations include combining rule-based filters with ML algorithms for improved accuracy.

2.2 SCOPE OF THE PROJECT

The scope of an automated plagiarism detection system is extensive, with applications across various domains, including academia, publishing, corporate environments, and research institutions. As digital content creation continues to grow, the need for an efficient, scalable, and precise plagiarism detection system becomes even more critical. The following key aspects define the scope of this plagiarism detection system:

1. Efficient and Automated Text Analysis:

The system automates the process of plagiarism detection by leveraging Natural Language Processing (NLP) and machine learning techniques to analyze and compare documents efficiently. This eliminates the need for manual comparison, which is often time-consuming and labor-intensive.

2. Multi-Format Document Processing:

The system supports various document formats, including PDF, DOCX, TXT, and online content, ensuring flexibility and compatibility with a wide range of digital texts. This enables seamless plagiarism detection across different sources.

3. High Accuracy in Similarity Detection:

By utilizing TF-IDF vectorization, cosine similarity, and advanced NLP techniques, the system can detect verbatim copying, paraphrased content, and partial similarities that might otherwise go unnoticed. This enhances the reliability and precision of plagiarism detection.

4. Cost-Effective Solution:

The implementation of an automated system significantly reduces the costs associated with manual plagiarism checking, especially in universities, research institutions, and publishing firms where large volumes of documents need to be processed regularly.

5. Enhanced Academic Integrity:

The system plays a crucial role in maintaining academic honesty, ensuring that students, researchers, and educators submit original work. By providing plagiarism reports, the system encourages ethical writing practices and discourages content duplication.

6. Real-Time and Scalable Analysis:

The system can handle large datasets and process multiple documents simultaneously, making it ideal for institutions that require plagiarism detection for thousands of papers or articles. It can also provide real-time analysis for immediate feedback.

7. Integration with Learning Management Systems (LMS):

The plagiarism detection system can be integrated into existing LMS platforms, research databases, and digital libraries, streamlining the workflow for educators, students, and administrators.

8. Predictive Analysis for Content Similarity Trends:

By collecting and analyzing plagiarism data, the system can identify patterns of academic dishonesty, track content reuse trends, and provide insights into evolving plagiarism techniques.

9. Intellectual Property Protection:

Beyond academia, businesses and legal firms can use the system to safeguard intellectual property rights, ensuring that copyrighted materials are not misused or duplicated without authorization.

10. Support for Multiple Languages:

Future enhancements may include multilingual plagiarism detection, allowing the system to analyze and compare content written in different languages, expanding its applicability to global research and publishing.

2.2.1 Problem Statement

Plagiarism in academic and professional content has become a growing concern in the digital age. With the vast availability of online resources and digital documents, it has become increasingly easy for individuals to copy content—either intentionally or unintentionally—without proper attribution. Traditional methods of detecting plagiarism, such as manual comparison or keyword matching, are time-consuming, error-prone, and insufficient when dealing with paraphrased or restructured text.

There is a pressing need for an automated, intelligent system that can analyze and compare multiple documents efficiently, detect not only exact matches but also semantic similarities, and generate a reliable similarity score. Such a system must handle different file formats, process textual data with consistency, and assist institutions in maintaining academic integrity and content originality.

The proposed solution aims to address these needs by designing a plagiarism detection system that:

- Supports .txt, .pdf, and .docx file formats
- Uses natural language processing (NLP) techniques to preprocess text
- Applies TF-IDF vectorization and cosine similarity to measure textual overlap
- Flags documents exceeding a predefined similarity threshold
- Generates structured reports to support educators and reviewers in decision-making

This system is designed to be scalable, efficient, and adaptable to institutional requirements, contributing to the broader effort of upholding ethical writing practices.

2.2.2 Existing System

Traditional plagiarism detection relies on manual verification or basic text-matching software that checks for exact matches between documents. Some widely used approaches include:

Manual Comparison: Educators or reviewers read and compare documents side by side to identify similarities.

Basic Keyword Matching: Some tools scan for identical words or phrases in different documents.

Internet-Based Search: Search engines are used to find copied content from online sources.

Rule-Based Systems: Simple algorithms compare documents based on predefined rules, such as word frequency or sequence matching.

2.2.3 Disadvantages of Existing System

Time-Consuming & Inefficient: Manually comparing documents is slow and impractical for large datasets.

Limited Accuracy: Basic keyword matching fails to detect paraphrased or restructured content.

No Contextual Understanding: Traditional systems lack Natural Language Processing (NLP) capabilities to analyze meaning and context.

Inability to Detect Cross-Language Plagiarism: Most existing methods cannot identify plagiarism in translated or multi-language content.

Scalability Issues: Manual and rule-based approaches struggle to handle a large volume of documents efficiently.

Lack of Actionable Insights: Simple matching systems do not provide detailed similarity reports or highlight specific copied sections.

Due to these limitations, there is a need for a more advanced, automated plagiarism detection system that leverages NLP, TF-IDF, and Cosine Similarity for accurate, efficient, and scalable text analysis.

2.4 PROPOSED SYSTEM

The proposed Automated Plagiarism Detection System aims to overcome the limitations of traditional plagiarism detection by integrating Natural Language Processing (NLP) and Machine Learning techniques. The system will efficiently analyze documents, detect similarities, and provide a structured similarity report with actionable insights.

The core functionalities of the system include:

Text Preprocessing: Removing stopwords, punctuation, and formatting inconsistencies for cleaner data.

Feature Extraction using TF-IDF: Converting textual data into numerical representations for similarity measurement.

Cosine Similarity Calculation: Determining the degree of similarity between documents using vector-based approaches**.**

Advanced NLP Techniques: Using spaCy and deep learning models to identify reworded or paraphrased content.

Comprehensive Database Matching: Comparing documents against a large database, including academic papers, web sources, and institutional repositories.

Automated Plagiarism Reports: Generating detailed similarity reports highlighting plagiarized sections.

Advantages Of Proposed System

High Accuracy: The system detects both exact and paraphrased plagiarism, improving reliability.

Efficiency & Speed: The automated system processes multiple documents simultaneously, significantly reducing verification time.

Scalability: Supports large-scale document analysis, making it ideal for universities, publishers, and corporations.

Contextual Understanding: NLP techniques allow the system to understand semantic similarities rather than just exact text matches.

Real-Time Detection: Provides instant plagiarism reports, enabling quick decision-making.

Multilingual Support: Capable of detecting plagiarism across multiple languages, expanding its applicability.

Detailed & Actionable Insights: Generates reports with highlighted sections, similarity percentages, and sources, making review easier.

Secure & Ethical: Helps maintain academic integrity by promoting original content creation and preventing intellectual property violations.

By leveraging Machine Learning and Natural Language Processing, this system outperforms traditional plagiarism detection methods and ensures greater accuracy, efficiency, and scalability in content verification.

2.4 REQUIREMENTS

2.4.1 Software Requirements

The software environment consists of all the platforms, tools, and packages necessary to build, run, and test the plagiarism detection system

Component	**Specification**
Operating System	Windows 10 / 11 or any Linux-based OS
Development Platform	Jupyter Notebook / VS Code / PyCharm
Programming Language	Python 3.8 or above
Interface (Optional)	Console-based or Web-based (for visualization)
Libraries / Packages	pandas, spaCy, scikit-learn, PyMuPDF (fitz), python-docx

. The system is built using open-source technologies, ensuring wide accessibility and ease of use.These tools and libraries enable efficient text extraction, natural language processing, vectorization, and similarity calculation. The choice of Python and open-source packages makes the project cost-effective and easy to maintain, while the flexibility of the development platforms allows for faster debugging and iteration.

2.4.2 Hardware Requirements

The hardware specifications listed below represent the minimum configuration required to effectively develop and run the Plagiarism Detection System. These requirements ensure that the system can handle tasks such as document processing, text vectorization, and similarity computation without performance bottlenecks, even when processing large numbers of documents.

Component	Specification
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Hard Disk	500 GB or more (for dataset storage and logs)
Display	Minimum 1024×768 resolution
System Type	64-bit Operating System

2.4.3 Functional Requirements

Functional requirements describe the expected behavior and operations of the system.

- The system should allow users to upload multiple documents in .pdf, .docx, and .txt formats.
- The system should automatically extract and preprocess text using NLP.
- It should calculate TF-IDF vectors for each document.
- The system should compute cosine similarity between document vectors.
- A report should be generated displaying file names, similarity percentages, and plagiarism status.
- Documents with similarity above the predefined threshold (e.g., 60%) should be flagged.

2.4.4 Non-Functional Requirements

- **Usability:** The system should be user-friendly and easy to operate, with potential for integration into a graphical or web interface.
- **Reliability:** The system should consistently provide accurate similarity results under repeated executions.
- **Performance:** The system should efficiently process small to moderately large sets of

documents without significant delay.

- **Scalability:** The system should support increasing numbers of documents and accommodate future enhancements such as semantic similarity or language support.
- **Supportability:** The system should be maintainable, using open-source Python libraries that are well-documented and widely supported.
- **Security:** Although not deployed online, the system should ensure local data privacy by not storing or exposing document content externally.

2.5 METHODOLOGY

2.5.1 Modules Name

1. File Handling and Text Extraction
2. Text Preprocessing using NLP
3. TF-IDF Vectorization
4. Cosine Similarity Computation
5. Result Evaluation and Reporting

2.5.2 Modules Explanation

1. File Handling and Text Extraction

This module is responsible for reading documents from a specified directory. It supports .txt, .pdf, and .docx file formats. Each document's text is extracted using appropriate libraries: built-in file handling for .txt, PyMuPDF (fitz) for .pdf, and python-docx for .docx files. The extracted content is stored in a structured format for further processing.

2. Text Preprocessing using NLP

After text extraction, this module cleans and prepares the text. It uses spaCy, a Natural Language Processing library, to convert all text to lowercase, remove stopwords and punctuation, and tokenize words. This ensures that only meaningful words are retained for analysis, improving the accuracy of similarity detection.

3. TF-IDF Vectorization

In this module, the cleaned text is transformed into numerical vectors using the TF-IDF (Term Frequency-Inverse Document Frequency) technique. TF-IDF assigns weight to each word in a document based on how common or unique it is relative to other documents. This helps capture the importance of words for comparison.

4. Cosine Similarity Computation

\ Once documents are converted into vectors, cosine similarity is calculated between every pair of documents. This similarity score indicates how close two documents are in terms of their content. A higher score (e.g., above 60%) suggests a stronger similarity, which could indicate potential plagiarism.

5. Result Evaluation and Reporting

In the final module, the system evaluates similarity scores for all document pairs. If a pair exceeds the predefined threshold (e.g., 60%), it is flagged as potentially plagiarized. The system then generates a structured report showing the file names, similarity percentage, and whether plagiarism was detected or not. This output helps users quickly identify issues and take appropriate action.

2.6 TECHNIQUES OR ALGORITHMS

Plagiarism Detection System is designed using a combination of Natural Language Processing (NLP) techniques and text similarity algorithms. The system mainly relies on three core components: text preprocessing, vectorization, and similarity computation.

2.6.1 Natural Language Processing (NLP) – Preprocessing

This step prepares the raw text extracted from documents for analysis. Using the spaCy library, the system performs:

- **Lowercasing:** Converts all text to lowercase for uniformity.
- **Stopword Removal:** Removes commonly used words that do not add meaning (e.g., “and”, “the”, “is”).
- **Punctuation Removal**: Eliminates commas, periods, and other non-alphabetic characters.
- **Tokenization:** Breaks the text into individual meaningful words or tokens.

This cleaning process ensures that only the most relevant textual data is retained for analysis.

2.6.2 TF-IDF (Term Frequency – Inverse Document Frequency)

Documents are quantitatively represented using TF-IDF according to the meaning of each word:

• Term Frequency (TF): Determines the frequency with which a word occurs in a given document.

• Inverse Document Frequency (IDF): Indicates a word's uniqueness across all of the dataset's documents.

When TF and IDF scores are combined, the weight of frequently used terms is decreased and unique and significant words are given more weight. Documents can be transformed into feature vectors for comparison with the use of TF-IDF.

2.6.3 Cosine Similarity

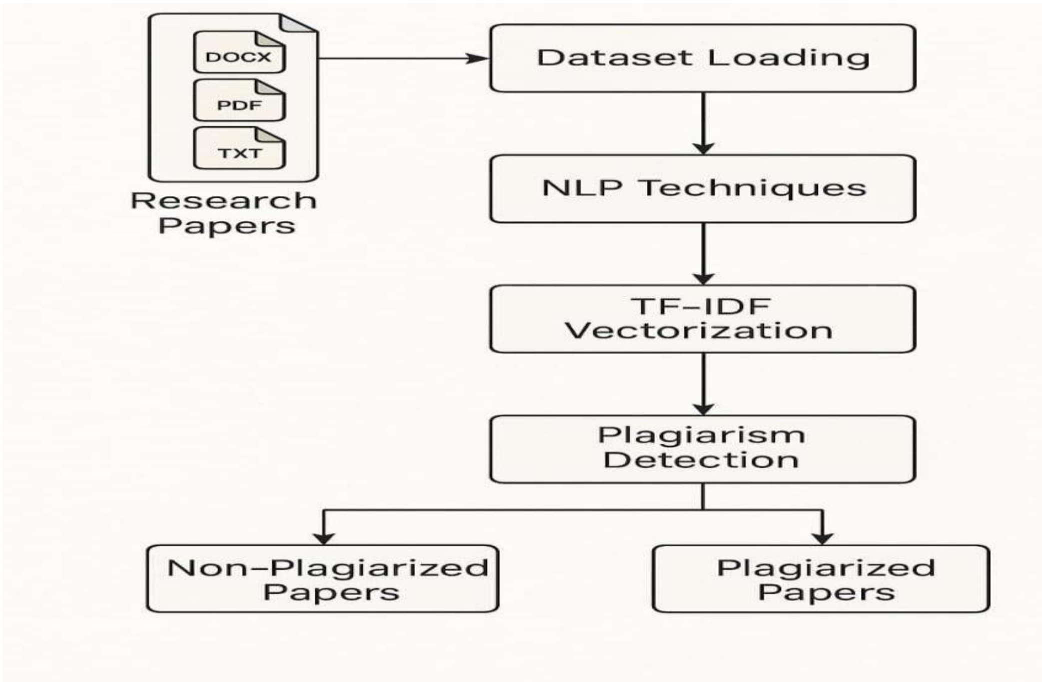
Cosine similarity is used to quantify the degree of similarity between documents after they have been transformed into TF-IDF vectors. The cosine of the angle formed by two vectors can be computed using this mathematical formula:

A number around 0 denotes little to no similarity, whereas a score of 1 (or 100%) denotes identical material.

The method can identify plagiarism even when rewording or paraphrasing is utilized since cosine similarity allows it to detect both precise and partial matches between documents.

CHAPTER-3
DESIGN AND DEVELOPMENT

3.1 SYSTEM ARCHITECTURE



3.1 System Architeture

The system architecture outlines a plagiarism detection pipeline for research papers in DOCX, PDF, and TXT formats. It starts with Dataset Loading, followed by NLP Techniques for text preprocessing. The processed text is then transformed using TF-IDF Vectorization to obtain numerical representations. These vectors are analyzed in the Plagiarism Detection phase, which classifies documents into Non-Plagiarized or Plagiarized Papers based on similarity scores.

3.2 GENERAL-UML DIAGRAMS
3.2.1 Use case Diagram

The Use Case Diagram outlines a user-driven plagiarism detection system composed of sequential functional components. It begins with the user uploading documents, after which the system performs text extraction to retrieve raw content. This content is then passed through text preprocessing techniques to prepare the data for analysis. The refined text is compared using cosine similarity, which calculates the degree of similarity between document vectors.

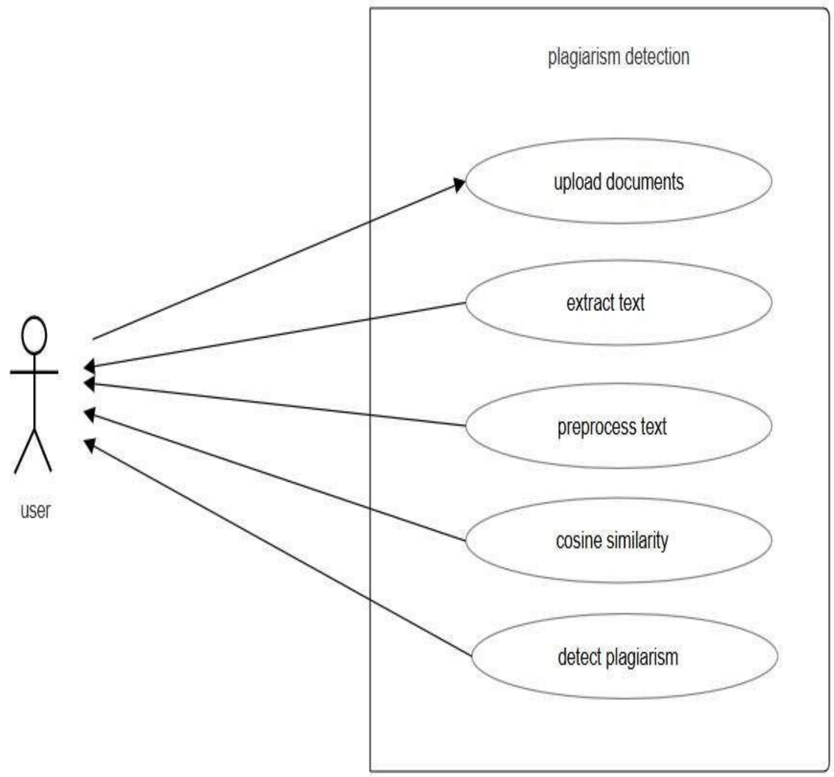


Fig. No. 3.2.1.1 Usecase Diagrams

Finally, based on these similarity scores, the system proceeds to the plagiarism detection phase, determining whether the content is original or copied.

3.2.2 Class Diagram

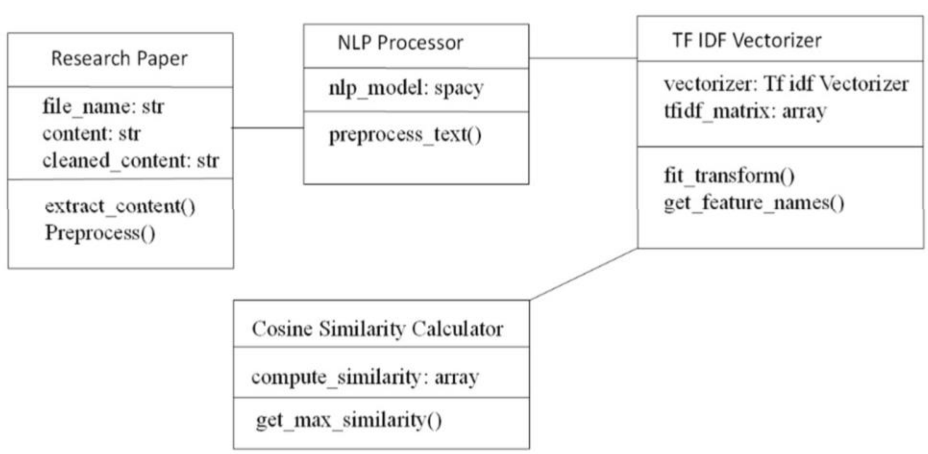


Fig.No-3.2.2.2 Class Diagram

The Class Diagram outlines the core components involved in a plagiarism detectionsystem using object-oriented design. It starts with the Research Paper class, which stores attributes like file_name, content, and cleaned_content, and includes methods such as extract_content() and Preprocess() for handling document input. The NLP Processor class utilizes the spacy model and provides the preprocess_text() function to clean and standardize text data. This processed text is then passed to the TF-IDF Vectorizer, which holds a TfidfVectorizer and generates the tfidf_matrix using fit_transform() and retrieves term features with get_feature_names(). Finally, the Cosine Similarity Calculator class computes similarity scores between documents through compute_similarity() and identifies the most similar documents using get_max_similarity().

3.2.3 Object Diagram

The object diagram illustrates a similarity computation workflow for research papers using NLP and vector-based analysis. It begins with two research paper objects, each encapsulating the file_name and file_content. These documents are passed to the NLP Processor, which utilizes the spacy model to apply core Natural Language Processing techniques such as tokenization, lemmatization, and stop-word remov

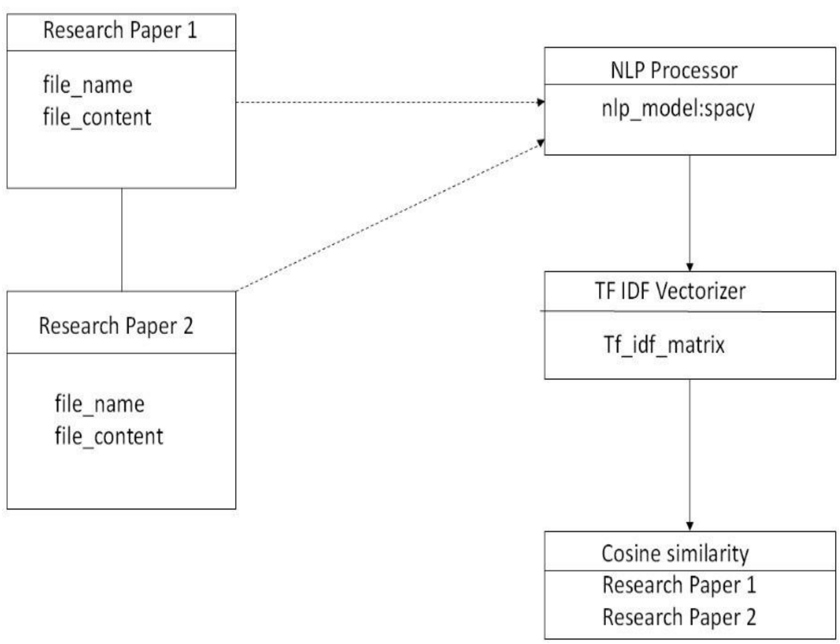


Fig.No-3.2.3.1 Object Diagram

The cleaned textual data is then forwarded to the TF-IDF Vectorizer, which generates a Tf_idf_matrix representing the term-weighted document vectors. These vectors are finally evaluated in the Cosine Similarity object, which calculates the similarity score between the two documents. The resulting similarity metric serves as the basis for determining textual closeness, commonly used in plagiarism detection and academic content comparison systems.

3.2.4 Component Diagram

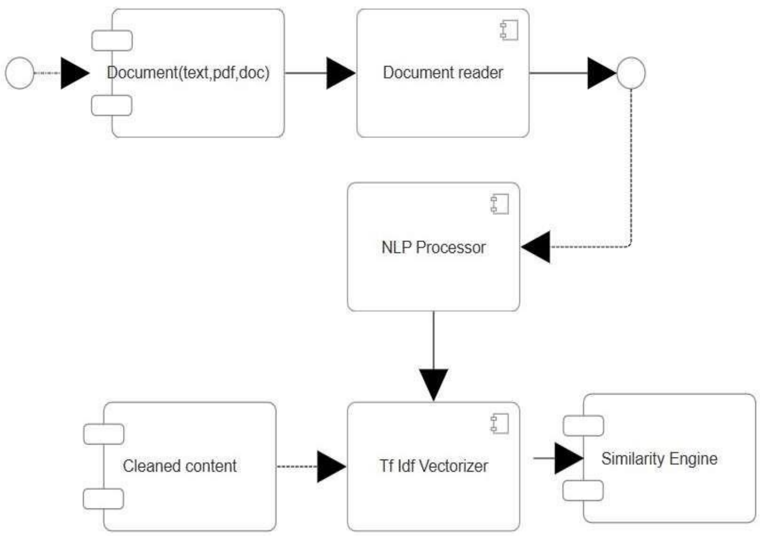


Fig.No-3.2.4.1 Component Diagram

The component diagram depicts a document similarity system for TXT, PDF, and DOC formats. It begins with the Document component, which feeds input to the Document Reader for extracting raw text. The extracted content is processed by the NLP Processor using techniques like tokenization, lemmatization, and stop-word removal to generate Cleaned Content. This content is transformed by the TF-IDF Vectorizer into weighted feature vectors. Finally, the Similarity Engine analyzes these vectors using cosine similarity to compute document similarity scores, enabling plagiarism detection and semantic comparison.

3.2.5 Deployment Diagram

The deployment diagram represents a plagiarism detection system across distributed nodes. The User Device (<<device>>) accesses the system via a browser or terminal interface. It connects to the Application Server (<<artifact>>), which runs the Plagiarism Detection Application. This application interacts with File Storage (<<artifact>>) containing Research Papers, and produces Results Output (<<artifact>>) holds Analysis Results returned to user.

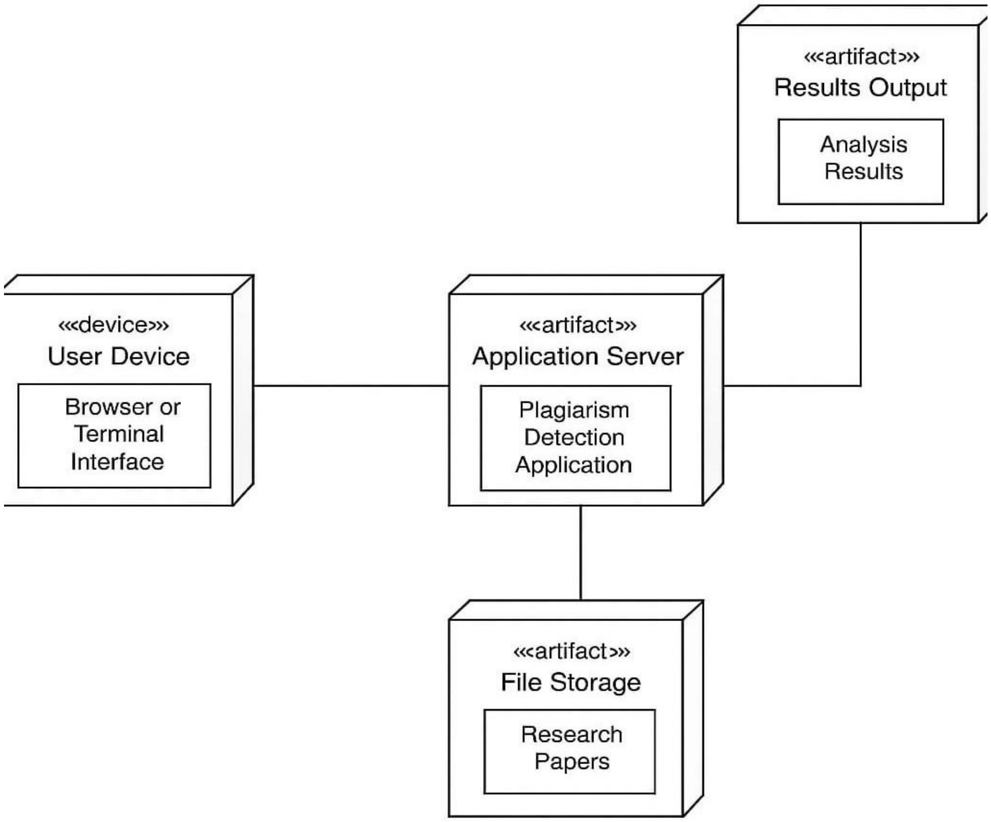


Fig.No-3.2.5.1 Deployment Diagram

3.2.6 Sequence Diagram

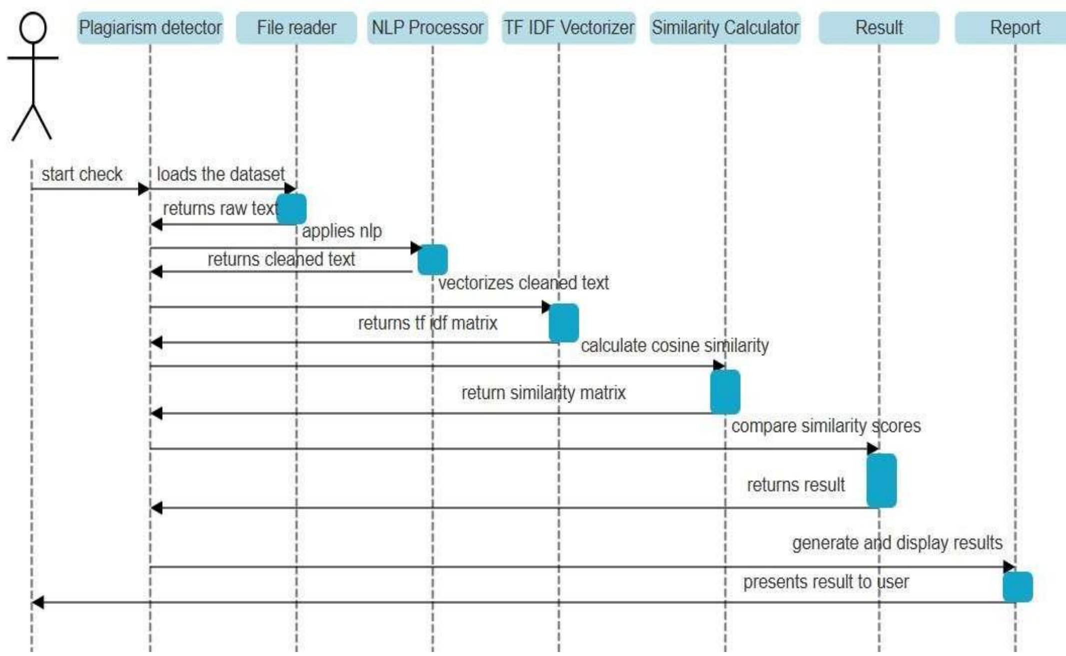


Fig.No-3.2.6.1 Sequence Diagram

The sequence diagram models the interaction flow within a plagiarism detection system. The process begins with the Plagiarism Detector initiating the check, prompting the File Reader to load the dataset and return raw text. The NLP Processor then applies preprocessing techniques, returning cleaned text. This is passed to the TF-IDF Vectorizer, which generates a TF-IDF matrix. The matrix is sent to the Similarity Calculator to compute cosine similarity, resulting in a similarity matrix. The Result component compares similarity scores and returns the final result, which is used by the Report module to generate and display results to the user.

3.2.7 Collaboration Diagram

The collaboration diagram outlines the interaction between system components in a plagiarism detection workflow. The process starts with the User Interface, which initiates file reading through the Document Reader. Extracted text is passed to the Text Preprocessor for cleaning. The preprocessed text is then fed into the TF-IDF Processor, which vectorizes the content.

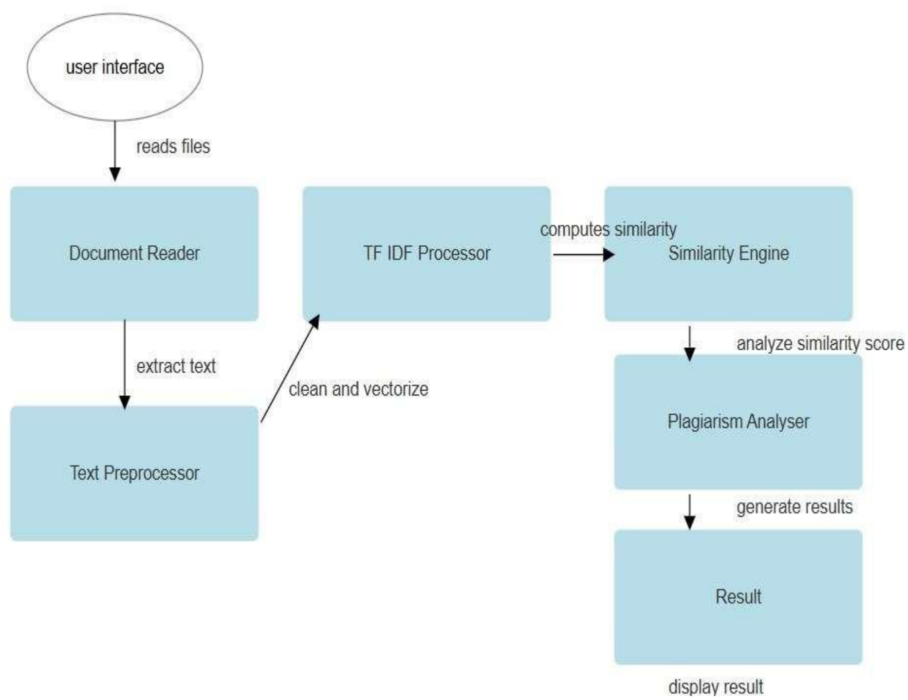


Fig.No-3.2.7.1 Collaboration Diagram

These vectors are used by the Similarity Engine to compute similarity scores. The Plagiarism Analyser evaluates these scores and forwards the findings to the Result module, which finally displays the results to the user.

3.2.8 State Diagram

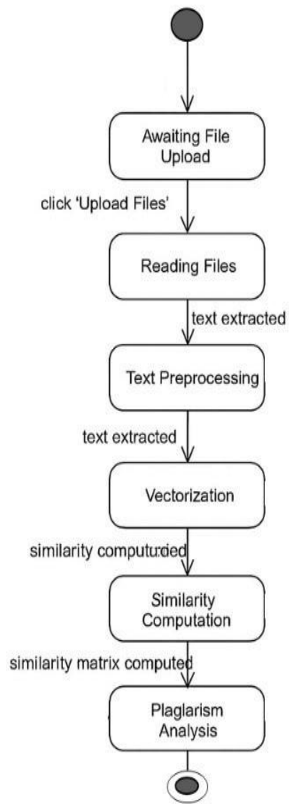


Fig.No-3.2.8.1 State Diagram

The State Chart Diagram represents the sequential states involved in the plagiarism detection workflow. The process initiates in the Awaiting File Upload state, transitioning to Reading Files once the user clicks 'Upload Files'. Upon successful text extraction, the system enters the Text Preprocessing state, preparing the content for analysis. The cleaned text then moves to the Vectorization stage, where it's numerically represented. Following this, Similarity Computation is performed to measure textual closeness, resulting in a computed similarity matrix. The final state, Plagiarism Analysis, uses this matrix to determine and report potential content duplication.

3.2.9 Activity Diagram

The Activity Diagram illustrates the logical flow of the plagiarism detection process. It begins with importing necessary libraries and loading the dataset. The text data is then preprocessed to prepare it for analysis.

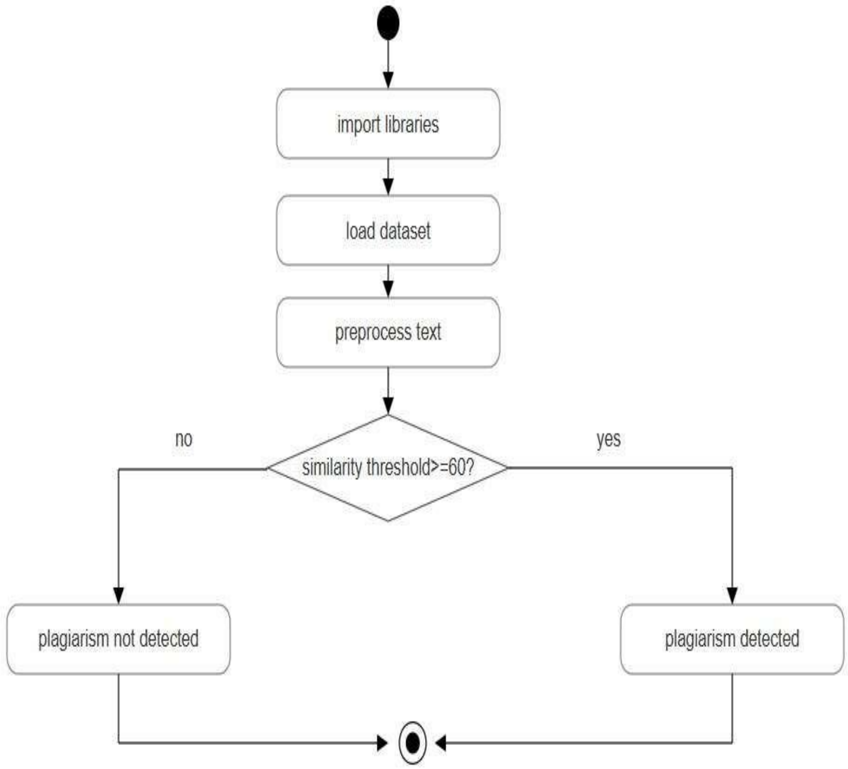
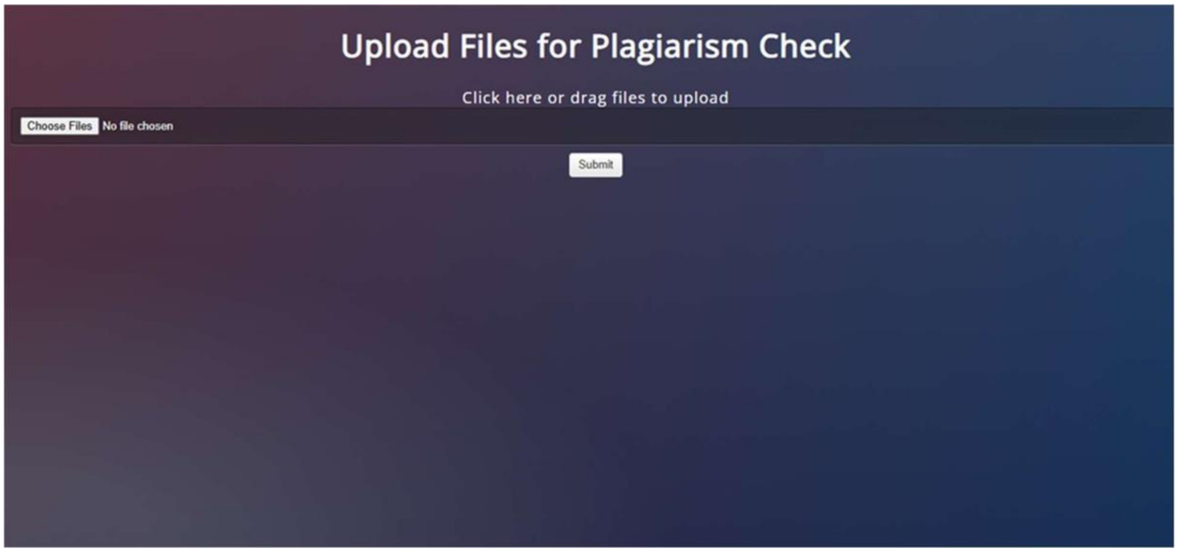


Fig.No-3.2.9.1 Activity Diagram

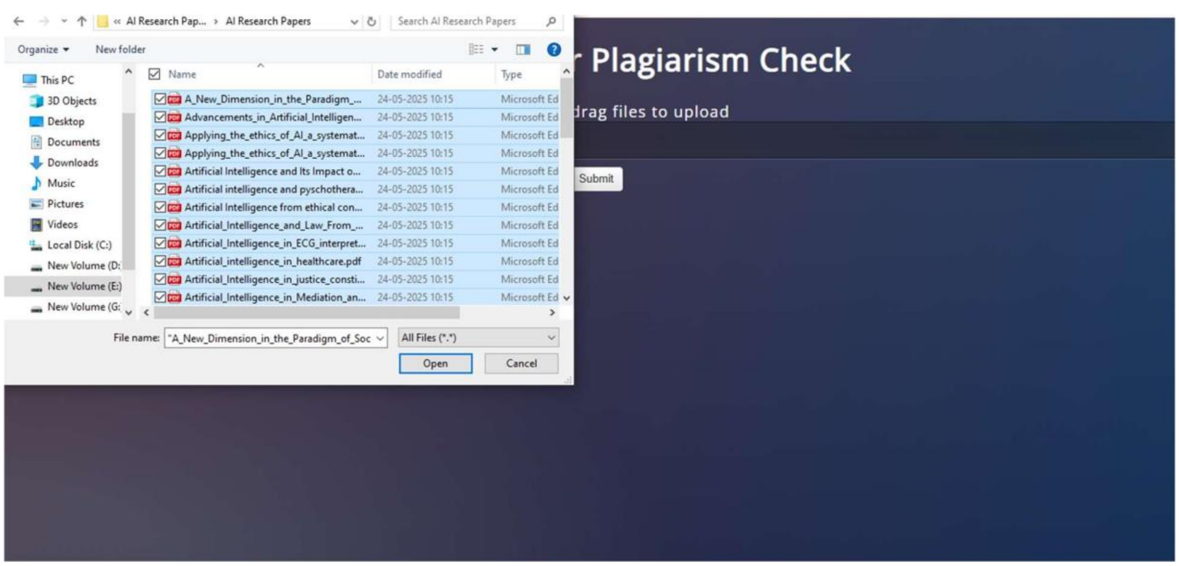
. A decision node checks whether the computed similarity threshold is greater than or equal to 60%. If the condition is met, the system transitions to Plagiarism Detected; otherwise, it concludes with Plagiarism Not Detected. The flow terminates after the outcome is established.

CHAPTER-4

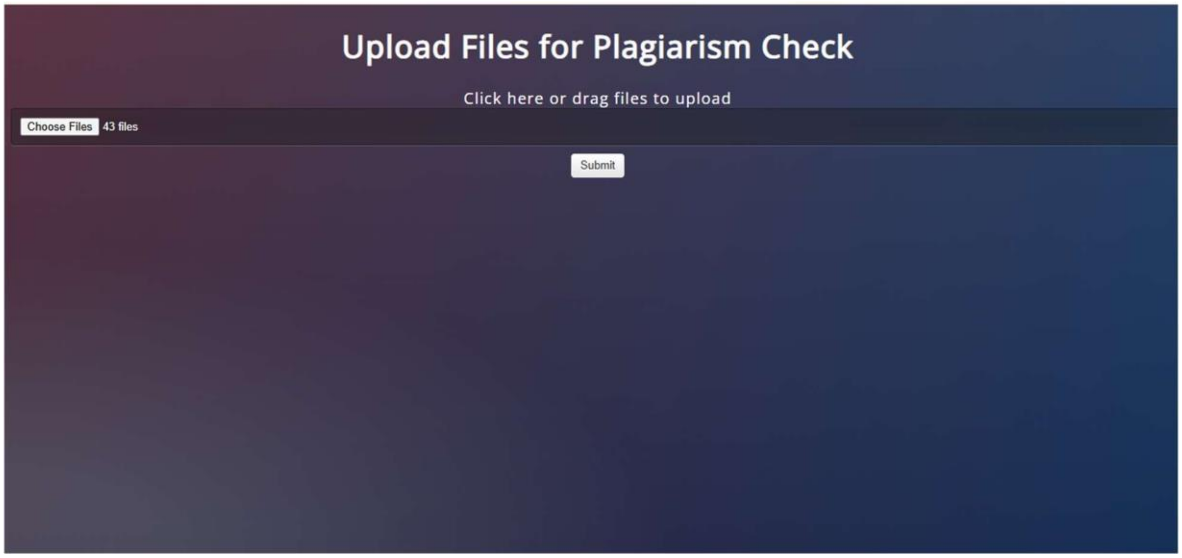
RESULTS AND DISCUSSION



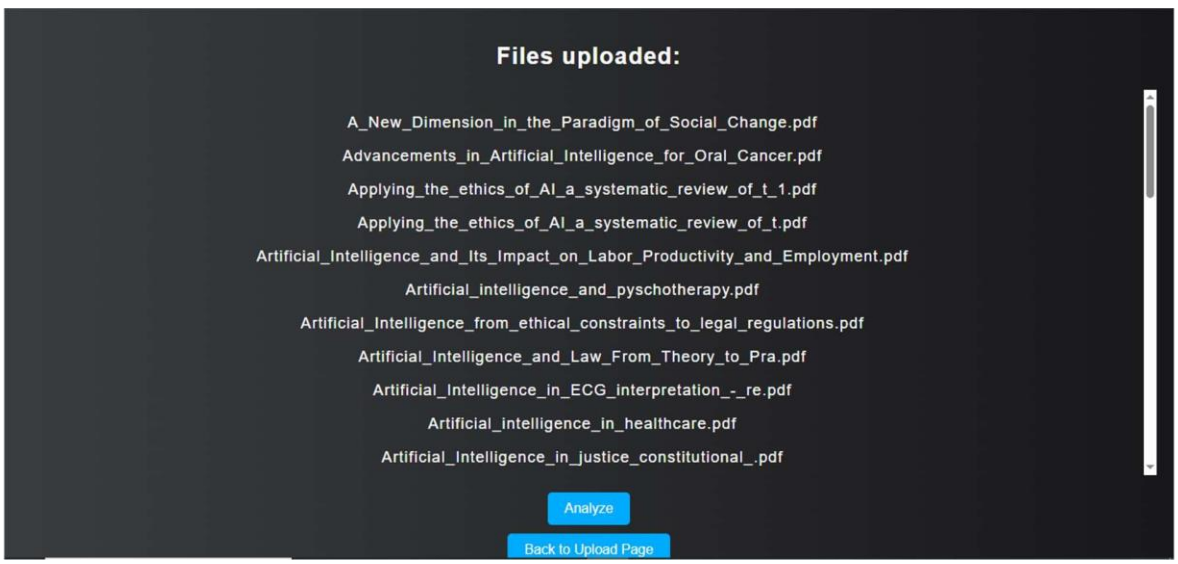
This is the file upload interface for the plagiarism detection system. Users can drag, drop, or select multiple documents for analysis and submit them for checking.



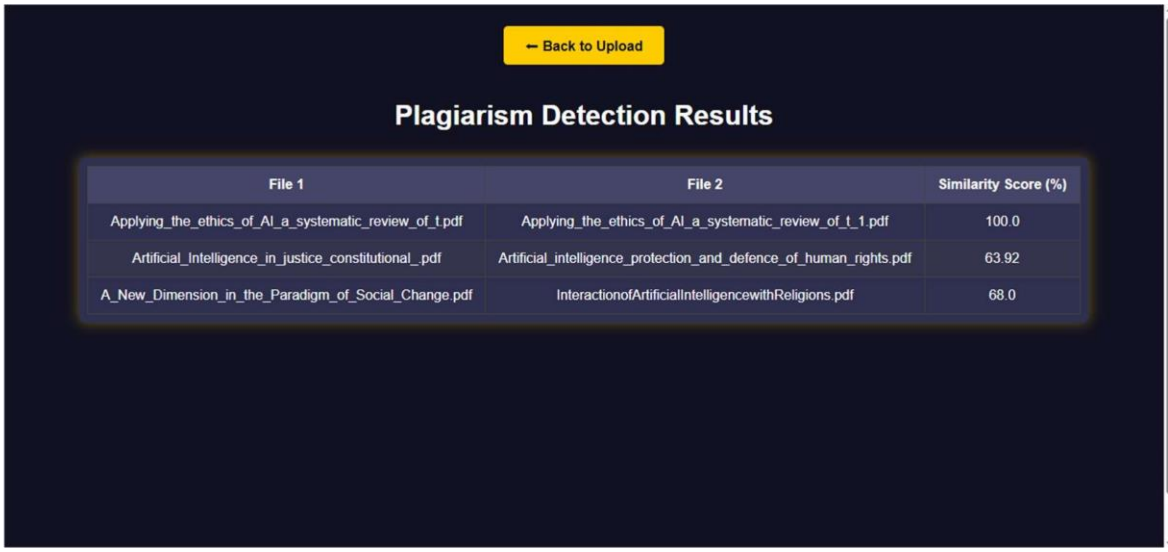
The interface displays a file upload module for plagiarism detection. Users can select multiple research papers from their system, as shown in the second image, and upload them using the "Choose Files" and "Submit" buttons to initiate the plagiarism check.



The interface displays the page where the no. of files uploaded for the plagiarism detection are shown and the by selecting the “submit” button it directs us to the page where all the files are displayed that are selected for plagiarism detection.



The interface allows users to upload multiple research papers in formats like PDF for plagiarism analysis. After submission, the system displays a list of uploaded files with options to proceed for analysis or return to the upload page.



The interface displays the plagiarism detection results for uploaded research papers. It shows a comparison table listing file names, matched counterparts, and corresponding similarity scores, helping users identify potential instances of content overlap across documents.

The Plagiarism Detection System was developed to identify similarities among textual documents. The system was tested using .txt, .pdf, and .docx files containing both original and plagiarized content. The results are generated after applying text extraction, preprocessing, vectorization using TF-IDF, and calculating cosine similarity between each document pair.

The output includes:

- A **similarity matrix** representing the cosine similarity percentages between documents.
- A **plagiarism report** listing all document pairs with a similarity score above the defined threshold (e.g., 60%).
- A separate listing of documents that did not cross the similarity threshold, confirming their uniqueness.

CHAPTER-5

CONCLUSION

5.1 CONCLUSION

The **Plagiarism Detection System** developed in this project provides an efficient and automated approach to identifying similarities between documents. By leveraging **Natural Language Processing (NLP)** techniques and **TF-IDF vectorization** with **cosine similarity**, the system accurately measures the degree of textual overlap between multiple files. It supports common document formats such as `.txt`, `.pdf`, and `.docx`, making it practical and versatile for academic and professional use.

Through modular design and reliable preprocessing methods, the system is capable of identifying both direct copying and moderately paraphrased content. It significantly reduces the manual effort involved in plagiarism checking and promotes ethical writing practices by ensuring content originality.

The implementation is built using Python and open-source libraries, which enhances its accessibility and adaptability. The system's structured reporting helps users make informed decisions based on similarity percentages.

In conclusion, this project successfully addresses the challenges of modern plagiarism detection by providing a fast, scalable, and accurate solution. It lays the foundation for future improvements such as semantic detection, cross-language comparison, and web-based integration, making it a valuable tool for institutions aiming to uphold academic integrity.

5.2 FUTURE ENHANCEMENMTS

1) Cross-Language Plagiarism Detection

Enhance the system to identify plagiarism between documents written in different languages using translation models and multilingual embeddings.

2) Integration with Online Databases

Allow the system to compare submitted documents against **online sources**, digital libraries, and research databases to catch plagiarism beyond the local dataset.

3) Semantic Similarity Detection

Implement advanced language models such as **BERT** or **RoBERTa** to capture the **semantic meaning** of sentences, enabling detection of paraphrased or restructured plagiarism more effectively.

REFERENCES

1) Sajid, M., Sanaullah, M., Fuzail, M., Malik, T. S., & Shuhidan, S. M. (2025). *Comparative analysis of text-based plagiarism detection techniques.* PLOS ONE, 20(4), e0319551. DOI:10.1371/journal.pone.0319551journals.plos.org.
2) Amirzhanov, A., Turan, C., & Makhmutova, A. (2025). *Plagiarism types and detection methods: A systematic survey of algorithms in text analysis.* Frontiers in Computer Science, 7:1504725. DOI:10.3389/fcomp.2025.1504725frontiersin.org.
3) Osman, A. H., Salim, N., Kumar, Y. J., & Abobieda, A. (2012). *Fuzzy Semantic Plagiarism Detection.* In: **Advanced Machine Learning Technologies and Applications**, CCIS 322, pp. 543–553. DOI:10.1007/978-3-642-35326-0_54researchgate.net.
4) Xian, J., Yuan, J., Zheng, P., Chen, D., & Yuntao, N. (2024). *BERT-Enhanced Retrieval Tool for Homework Plagiarism Detection System.* arXiv:2404.01582arxiv.org.
5) Soori, H., Prilepok, M., Platos, J., & Snášel, V. (2016). *Semantic and Similarity Measure Methods for Plagiarism Detection of Students' Assignments.* Proc. 2nd Afro-European Conference for Industrial Advancement (AECIA), AISC 427, pp. 117–125link.springer.com.
6) Kundu, R., & Karthik, K. (2017). *Contextual plagiarism detection using latent semantic analysis.* Int. Research Journal of Advanced Engineering and Science, 2(1), 214–217irjaes.com.
7) Kulkarni, S., Mishra, P., & Yadav, R. (2020). *Semantic plagiarism detection system.* IRJET, 7(5), 534–541irjet.net.
8) Lee, S.-J., Wu, C.-H., Liu, C.-F., Lin, P.-R., Wu, Y.-L., & Hsu, F.-K. (2021). *Using word semantic concepts for plagiarism detection in text documents.* Information Retrieval Journal, 24, 298–321. DOI:10.1007/s10791-021-09394-4researchgate.net.
9) Moravvej, S. V., Mousavirad, S. J., Oliva, D., & Mohammadi, F. (2023). *A novel plagiarism detection approach combining BERT-based word embedding, attention-based LSTMs and an improved differential evolution algorithm.* arXiv:2305.02374arxiv.org.
10) Davoodifard, M. (2018). *Automatic detection of plagiarism in writing.* The Forum, 56(2), 54–60files.eric.ed.gov.